Right-sizing agent — implementation guide (mulat)

Date: 2026-11-07

This document explains the **second specialist agent** (right-sizing): which files changed, how data flows, and how it mirrors the node health agent.

---

1. Role of the right-sizing agent

Input	What the agent does
Supervisor sets `targets` includes `right_sizing`	LangGraph runs `right_sizing_node`
User request + optional `hints.namespace`	Collect **requests/limits** (Kubernetes) + **CPU/memory usage** (Prometheus)
Output	One **finding** with `data` (raw tool JSON) and `message` (LLM summary)

The agent **does not** apply new requests/limits. It only **reads** and compares (per Tool_API.md and architecture doc).

---

2. Files changed or added

2.1 Tool layer (`src/mulat/tools/right_sizing/`)

File	Purpose
`promql.py`	Builds PromQL strings for CPU and memory (Tool_API patterns)
`get_requests_limits.py`	`get_requests_limits_impl()` — lists pods, parses container resources
`query_cpu_usage.py`	`query_cpu_usage_impl()` — Prometheus instant query
`query_memory_usage.py`	`query_memory_usage_impl()` — Prometheus instant query

Each `*_impl` returns the standard envelope: `{ok, tool, data, truncated, error}`.

2.2 Client layer

File	Change
`clients/prometheus.py`	Added `query_instant_safe()` for tool-friendly error handling
`clients/kubernetes.py`	Unchanged — used by `get_requests_limits_impl`

2.3 Registry (`tools/registry.py`)

Change
Added `@tool` wrappers: `get_requests_limits`, `query_cpu_usage`, `query_memory_usage`
`sizing_tools = [get_requests_limits, query_cpu_usage, query_memory_usage]`

Registry is for **future ReAct**; the live path calls `*_impl` directly from the agent (same as node health).

2.4 Orchestration

File	Change
`orchestration/agents/right_sizing.py`	`collect_right_sizing_data()` + `right_sizing_node(state, llm)`
`orchestration/graph.py`	`right_sizing_step` node; edges `node_health → right_sizing → END`
`prompts/right_sizing.md`	Already existed — used for LLM summary

2.5 Configuration (`.env`)

Variable	Used by
`K8S_API_SERVER`, `K8S_TOKEN`	`get_requests_limits`
`PROMETHEUS_URL`	`query_cpu_usage`, `query_memory_usage`
`PROMETHEUS_QUERY_TIMEOUT_SECONDS`	HTTP timeout

---

3. Step-by-step execution flow

Step 1 — HTTP trigger

`POST /ask` → `graph.invoke(initial, config)` (see CALL_FLOW.md).

Step 2 — Supervisor

`supervisor_node_llm` or keyword supervisor sets `state["targets"]`.

Examples:

- Full check: `["node_health", "right_sizing", "pod_security"]`
- Sizing-only keywords: `["right_sizing"]` (cpu, prometheus, right-size in request)

Step 3 — Node health (if in targets)

Runs before right-sizing on the current graph edge order. If not in targets, returns `{}`.

Step 4 — Right-sizing node

```
right_sizing_node(state, llm)
  if "right_sizing" not in targets → return {}
  payload = collect_right_sizing_data(state)
    → get_requests_limits_impl(...)
    → query_cpu_usage_impl(stat=p95, lookback=7d)
    → query_memory_usage_impl(stat=max, lookback=7d)
  message = LLM(prompt right_sizing.md + payload JSON)
  return { findings: [{ agent, message, data: payload }] }
```

`findings` append to graph state (`operator.add`).

Step 5 — Response

`/ask` maps final state to `RunResponse`; `detail` may show first finding message.

---

4. Concept: three tools

4.1 `get_requests_limits` (Kubernetes)

- **API:** `GET /api/v1/pods` or namespaced pods
- **Data:** per-container `cpu_request_m`, `cpu_limit_m`, `mem_*`, `missing` fields
- **Errors:** `FORBIDDEN`, `UNAVAILABLE` via RBAC / API

4.2 `query_cpu_usage` (Prometheus)

- **API:** `GET {PROMETHEUS_URL}/api/v1/query`
- **PromQL:** `quantile_over_time(0.95, rate(...)[7d:5m])` pattern (see `promql.py`)
- **Data:** `series[]` with `value_m` (millicores)
- **Errors:** `PROMETHEUS_ERROR`, `NO_DATA`

4.3 `query_memory_usage` (Prometheus)

- **Metric:** `container_memory_working_set_bytes`
- **Stat:** default `max` over `7d`
- **Data:** `value_mib` per container

Agent compares **requests** from (1) with **usage** from (2) and (3); LLM writes the narrative.

---

5. Graph wiring

```
START → supervisor → node_health → right_sizing → END
```

Each node checks `targets` and **no-ops** if its agent name is missing.

Future: `Send()` parallel fan-out from supervisor; synthesizer and approval nodes.

---

6. Testing

1. Set `.env`: `PROMETHEUS_URL=http://<host>:<port>` (Service port 80 or NodePort). 2. Set Kubernetes token/server (same as node health). 3. `POST /ask`:

```
{
  "message": "Compare CPU and memory requests with actual usage in namespace prod",
  "hints": { "namespace": "prod" }
}
```

4. Inspect response / logs; finding `data` should contain three tool envelopes.

---

7. Related documents

Document	Contents
Tool_API.md	Tool contracts and PromQL
CALL_FLOW.md	FastAPI + LangGraph
DIRECTORY_STRUCTURE.md	Project layout